Toward Adaptive and Robust Machine Learning based Network Intrusion Detection Systems

Omar Anser

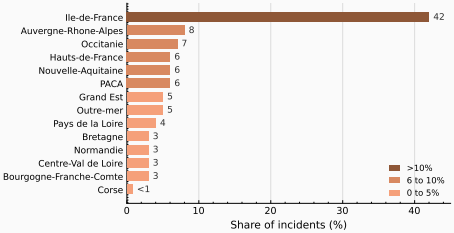
April 27, 2026

SnT — University of Luxembourg, Luxembourg

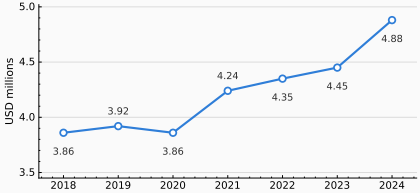
Introduction

Why this topic matters

- Modern societies and public services increasingly rely on computer networks $\longrightarrow$ Higher exposure to cyber incidents.
- ANSSI handled **4,386** security events in 2024 (**+15%** vs. 2023), including **1,361** confirmed incidents, i.e., nearly **4** per day.
- **$4.88M** average global cost of a data breach in 2024.
- **35%** of reported cyber incidents in the French health sector affected patient care.



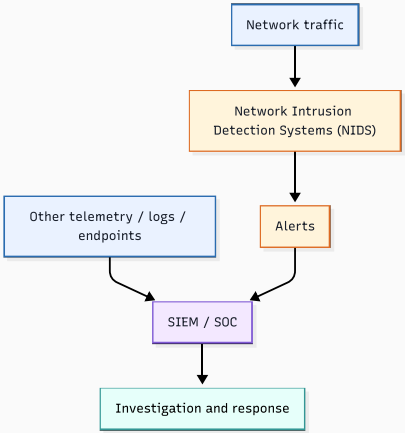
Source: **ANSSI, Panorama de la cybermenace 2024**



Source: **IBM Security, Cost of a Data Breach Report 2024**

Where NIDS fits in cyber defense

- Continuous monitoring to manage cyber risk
- SOCs and SIEM platforms centralize this monitoring
- Network traffic gives direct visibility into system communications
- NIDS turn traffic into alerts for investigation and response



⇒ **NIDS is one analytical layer inside the broader monitoring and response chain.**

Refs: NIST SP 800-86 (2006); Scarfone & Mell, Guide to Intrusion Detection and Prevention Systems (2007).

From signatures to ML-based NIDS

Rule-based NIDS

- Tools: **Snort**, **Suricata**, **Zeek**
- Detect only known attack patterns or protocol violations, leaving unknown ones undetected.
- Limited when payload content is encrypted

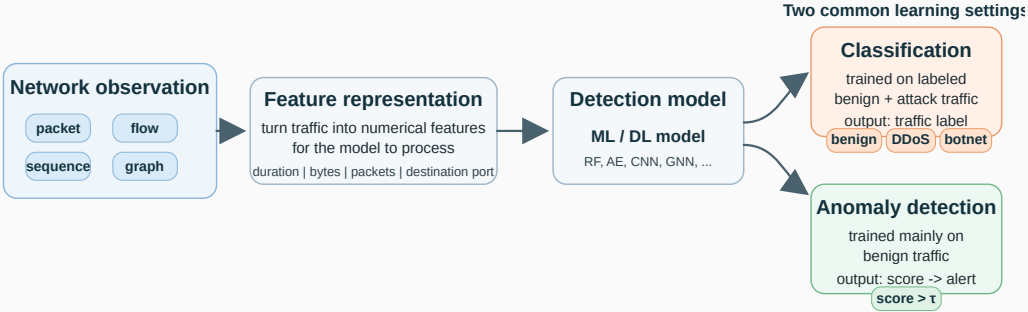
ML-based NIDS

- Example: **Random Forest (RF) on flows**
- Learn traffic patterns from historical data to generalize beyond exact rules.
- Can still detect suspicious behavior in encrypted traffic

Operational reality: mostly hybrid. For example, a SOC may use **Suricata/Zeek** for known alerts and **ML on flows** to flag suspicious HTTPS command-and-control behavior.

Refs: Snort [Roesch, 1999]; Zeek [Paxson, 1999]; Suricata (OISF); ML for NIDS [Sommer & Paxson, 2010]; encrypted traffic [Anderson & McGrew, 2016]; SANS Detection & Response Survey (2024).

ML-based NIDS as a prediction task

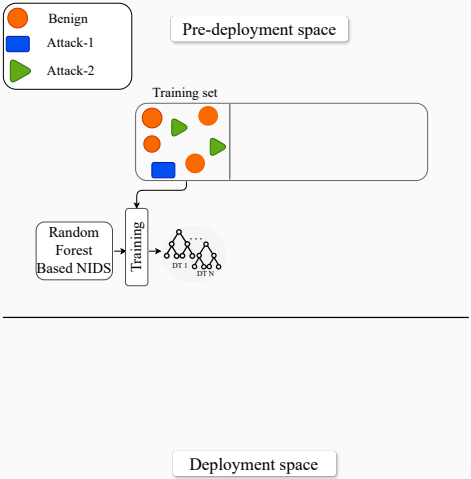


⇒ **This common ML setting leads to the standard train → validate → test → deploy workflow.**

Refs: Sommer & Paxson (2010); Buczak & Guven (2016).

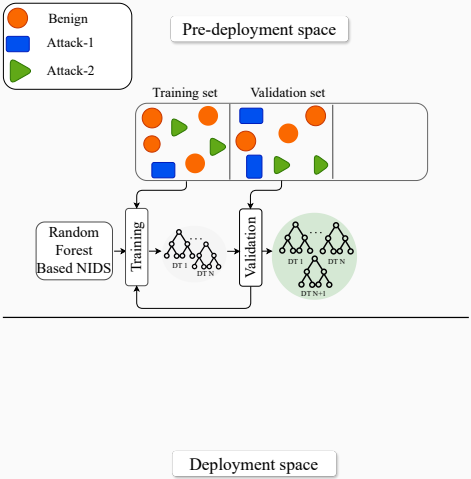
ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.



ML-based NIDS Workflow: Train → Validate → Test → Deploy

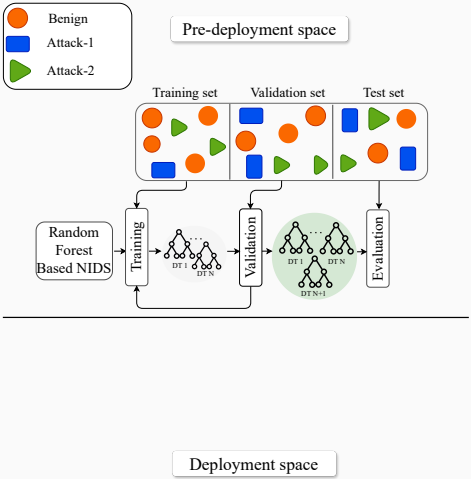
- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters**).



Hyperparameters: settings chosen before training that influence model behavior.
Example: In RF, too few trees can give unstable predictions, while too many can increase computation and overfit the training data.

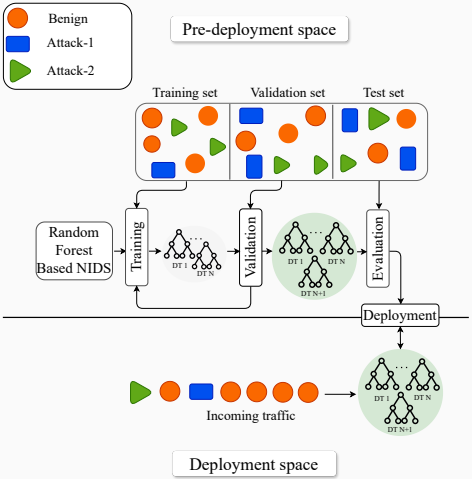
ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters**).
- A final **test** set, $\mathcal{D}_{\text{test}}$, measures performance on previously unseen data.



ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters**).
- A final **test** set, $\mathcal{D}_{\text{test}}$, measures performance on previously unseen data.
- The selected model is then **deployed** to monitor live traffic.



How ML has been used to improve NIDS

- Better detection performance $\longrightarrow$ BotMiner, Kitsune
  e.g., DDoS, botnet, web attacks
- Richer traffic representations $\longrightarrow$ Flow-MAE, NetVigil
  e.g., packet, flow, sequence, graph
- More practical deployment $\longrightarrow$ AOC-IDS, ENIDrift
  e.g., lightweight, online, real-time detection
- Better use of limited labels $\longrightarrow$ Active-SVDD, INSOMNIA
  e.g., active learning, semi-supervised learning, and pseudo-labeling
- Better robustness in practice $\longrightarrow$ CADE, OWAD
  e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation

Refs: BotMiner [Gu et al., 2008]; Kitsune [Mirsky et al., 2018]; Flow-MAE [Hang et al., 2023]; NetVigil [Hsieh et al., 2024]; AOC-IDS [Zhao et al., 2024]; ENIDrift [Wang, 2022]; Active-SVDD [Gornitz et al., 2009]; INSOMNIA [Pierazzi et al., 2021]; CADE [Yang et al., 2021]; OWAD [Hu et al., 2023].

How ML has been used to improve NIDS

- Better detection performance $\longrightarrow$ BotMiner, Kitsune
  e.g., DDoS, botnet, web attacks
- Richer traffic representations $\longrightarrow$ Flow-MAE, NetVigil
  e.g., packet, flow, sequence, graph
- More practical deployment $\longrightarrow$ AOC-IDS, ENIDrift
  e.g., lightweight, online, real-time detection
- Better use of limited labels $\longrightarrow$ Active-SVDD, INSOMNIA
  e.g., active learning, semi-supervised learning, and pseudo-labeling
- **Better robustness in practice $\longrightarrow$ CADE, OWAD**
  e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation

Refs: BotMiner [Gu et al., 2008]; Kitsune [Mirsky et al., 2018]; Flow-MAE [Hang et al., 2023]; NetVigil [Hsieh et al., 2024]; AOC-IDS [Zhao et al., 2024]; ENIDrift [Wang, 2022]; Active-SVDD [Gornitz et al., 2009]; INSOMNIA [Pierazzi et al., 2021]; CADE [Yang et al., 2021]; OWAD [Hu et al., 2023].

Focus of this thesis: robustness in ML-based NIDS

Robustness in this thesis

- Good performance on one fixed benchmark split is not sufficient to claim robustness.
- Before deployment, robustness depends on evaluation conditions that reveal detector weaknesses.
- After deployment, robustness means remaining effective when traffic and attack patterns evolve.

$\Rightarrow$ **ML-based NIDS robustness is studied both before deployment, through the evaluation process, and after deployment, through adaptability to evolving traffic.**

Roadmap of the presentation

Each part follows the same logic

State of the art → Limitations → Proposed contribution

Part I Evaluation

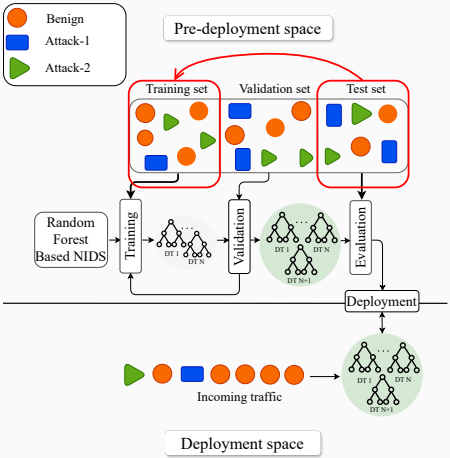
- Anser et al., *"TATA: Benchmark NIDS Test Sets Assessment and Targeted Augmentation,"* ESORICS 2025 (*main track*).

Part II Adaptation

- Anser et al., *"Auto-tuning of Hyper-parameters for Detecting Network Intrusions via Meta-learning,"* NOMS 2023 (*AnNet workshop*).
- Anser et al., *"Automated Machine Learning Configuration to Learn Intrusion Detectors on Attack-Free Datasets,"* LCN 2024 (*short paper*).
- Anser et al., *"An Empirical Study on Configuration Robustness of Unsupervised IDS,"* NOMS 2026 (*main track, to appear*).

Part I: TATA

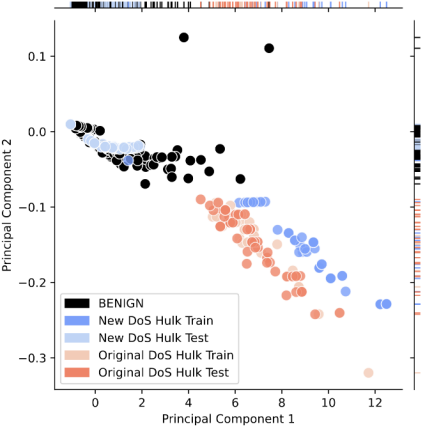
Introduction



$\Rightarrow$ **This part focuses on $\mathcal{D}_{\textbf{test}}$ and its relation to $\mathcal{D}_{\textbf{train}}$.**

Benchmark NIDS datasets

- A single source dataset partitioned into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$.
- Reported evaluation accuracies $>$ **90%** in published works on public benchmarks (IDS2017, UNSW-NB15...) $\rightarrow$ solved problem or benchmark-induced illusions?
- A near-complete overlap of DoS-Hulk flows in IDS2017's $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$[1].



Overlap between $\mathcal{D}_{\text{test}}$ and $\mathcal{D}_{\text{train}}$ does not prevent overfitting $\rightarrow$ it can hide it.

[1] Flood et al., Bad Design Smells in Benchmark NIDS Datasets, IEEE EuroS&P 2024.

RQ2 — Targeted Augmentation

Given a **too-easy** initial $\mathcal{D}_{\text{test}}$, can we apply systematic augmentation to generate new, **realistic**, and **challenging** traffic that is specifically designed to expose the hidden weaknesses of the NIDS?

Research Questions

RQ1 — Test-set Assessment

How can we create a numerical score or a set of metrics to objectively measure the difficulty of a $\mathcal{D}_{\text{test}}$ for a NIDS?

RQ2 — Targeted Augmentation

Given a **too-easy** initial $\mathcal{D}_{\text{test}}$, can we apply systematic augmentation to generate new, **realistic**, and **challenging** traffic that is specifically designed to expose the hidden weaknesses of the NIDS?

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			

Neuron coverage: Fraction of neurons whose activation exceeds a preset threshold at least once across the test set; higher → explore more of the model's internal logic.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			
Surprise coverage			

Surprise coverage: Measures how broadly the test set differs from what the model saw in training; higher → the test set includes highly surprising inputs.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			
Surprise coverage			
Mutation testing			

Mutation testing: Mutates the model and/or its training setup (e.g., weights, architecture, or hyperparameters), then checks if the test set can distinguish the mutant from the original model.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage	✓	✗	Model access (🔓) + NN-only ( ) + Ignores train–test alignment (📏)
Surprise coverage	✓	✗	🔓 + + Proximity-only focus (🎯)
Mutation testing	✓	✓	🔓 + 📏 + Model-change sensitive (🛠) Retrain per dataset (🔄) + Synthetic data (🧪)
Manual (Flood et al., 2024)	✗	✓	Unguided augmentation (👆)

G1 Model-related gaps **G2** Shortcomings in assessing $\mathcal{D}_{\text{test}}$ → RQ1 **G3** Weaknesses in $\mathcal{D}_{\text{test}}$ augmentation workflows → RQ2

TATA (Test-set Assessment & Targeted Augmentation)

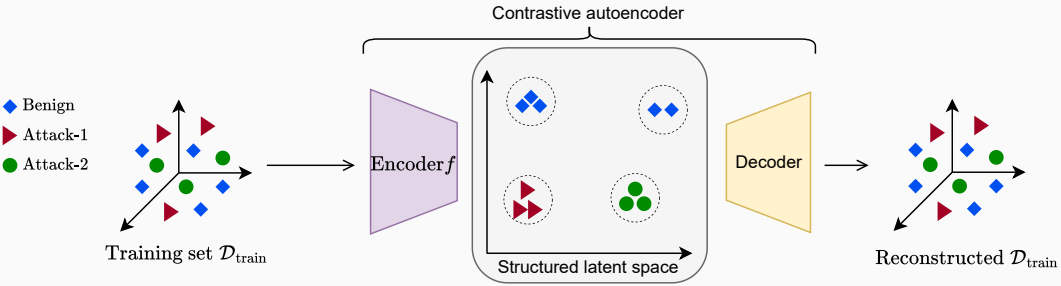
Goal: Close the gaps identified in related work and answer the research questions.

1. **Preliminary Phase** G1 : + +
   Approximate $\mathcal{D}_{\text{train}}$'s decision boundary while being model agnostic

2. **Phase 1 – Assesses $\mathcal{D}_{\text{test}}$'s quality (RQ1)** G2 : +
   Define three metrics (**Proximity**, **Diversity**, and **Scarcity**) to assess the **alignment** of $\mathcal{D}_{\text{test}}$ with respect to $\mathcal{D}_{\text{train}}$'s distribution.

3. **Phase 2 – $\mathcal{D}_{\text{test}}$ Augmentation (RQ2)** G3 : + +
   Reinforcement learning (RL) guides a testbed to generate **realistic traffic** that fills uncovered metric gaps, producing **a more challenging** $\mathcal{D}_{\text{test}}$.

Preliminary Phase: Approximating $\mathcal{D}_{\text{train}}$'s decision boundary



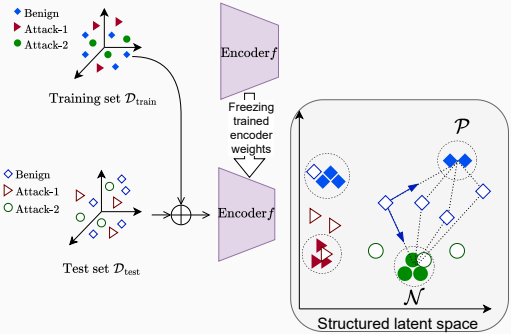
- Decision boundaries $\approx$ cluster borders in the latent space; these are captured in the encoder's weights.

Phase 1: Test-set Assessment

- Encode $\mathcal{D}_{\text{train}}$ & $\mathcal{D}_{\text{test}} \rightarrow \mathcal{Z}_{\text{test}}$ is located relative to $\mathcal{Z}_{\text{train}} \rightarrow$ we now work entirely in latent space $(\mathcal{Z}_{\text{train}}, \mathcal{Z}_{\text{test}})$.

- For each test embedding z, we identify:
  - $P(z)$: the nearest **same-label cluster** (*positive cluster*)
  - $N(z)$: the nearest **different-label cluster** (*negative cluster*)

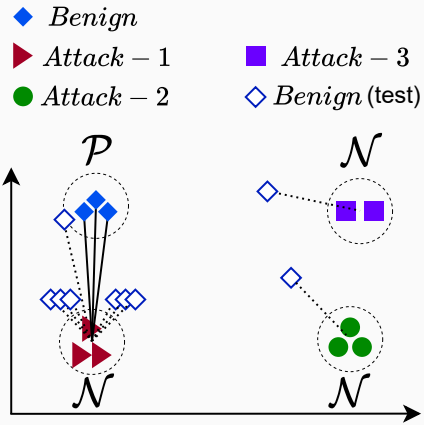
- Define proximity, diversity, and scarcity per $P(z)$; then aggreate.



Proximity $P \in [0, 1]$, higher = nearer to different-label boundaries

- Quantifies $\mathcal{Z}_{\text{test}}$ closeness to $N(z)$; high proximity $\Rightarrow$ closer to a different label decision boundary $\Rightarrow$ harder cases.

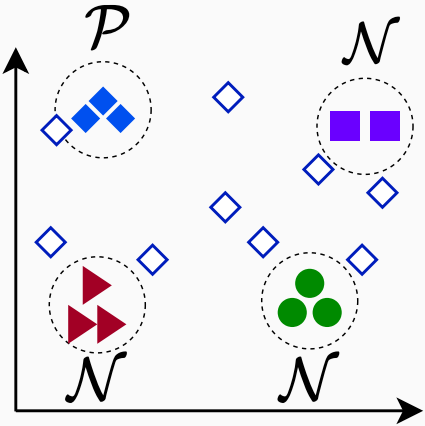
- P = **max** one-sided Kolmogorov–Smirnov statistic (over clusters): compares $\mathcal{Z}_{\text{test}}$ vs. $\mathcal{Z}_{\text{train}}$ distances to $N(z)$



Diversity $D \in [0, 1]$, higher = broader, less redundant coverage

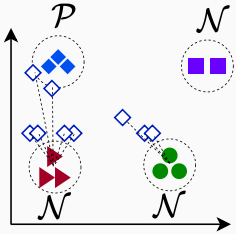
- Assesses the variety of $\mathcal{Z}_{\text{test}}$; higher diversity $\Rightarrow$ minimal redundancy $\Rightarrow$ broader coverage of the latent space.

- D = **Average** of normalized **Vendi Scores** (over clusters)

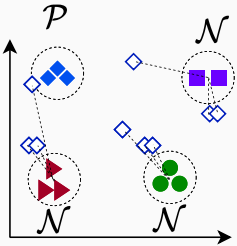


Scarcity $S \in [0, 1]$ higher = boundaries challenged more evenly

- Evaluates how uniformly $\mathcal{Z}_{\text{test}}$ are distributed among all potential negative clusters (i.e., decision boundaries).
- High scarcity $\Rightarrow$ a uniform spread $\Rightarrow$ all boundaries are exercised $\Rightarrow$ a more robust test set.
- S = **Average** of the **(1 - Gini coefficient)** (over clusters)



Bad scarcity



Optimal scarcity

Which approaches can augment the test set?

1. GANs / diffusion models.

Which approaches can augment the test set?

1. ~~GANs / diffusion models.~~
   - Synthetic generation may break protocol semantics $\Rightarrow$ unrealistic flows.

Which approaches can augment the test set?

1. ~~GANs / diffusion models.~~
   - Synthetic generation may break protocol semantics $\Rightarrow$ unrealistic flows.
2. Network simulators (e.g., NS-3, OMNeT++)

Which approaches can augment the test set?

1. ~~GANs / diffusion models.~~
   - Synthetic generation may break protocol semantics $\Rightarrow$ unrealistic flows.
2. ~~Network simulators (e.g., NS-3, OMNeT++)~~
   - Models protocol interactions, offering more realism.
   - Remaining realism gap vs. real hardware and user behavior.

Which approaches can augment the test set?

1. ~~GANs / diffusion models.~~
   - Synthetic generation may break protocol semantics $\Rightarrow$ unrealistic flows.
2. ~~Network simulators (e.g., NS-3, OMNeT++)~~
   - Models protocol interactions, offering more realism.
   - Remaining realism gap vs. real hardware and user behavior.
3. Using a **configurable live testbed**.

Which approaches can augment the test set?

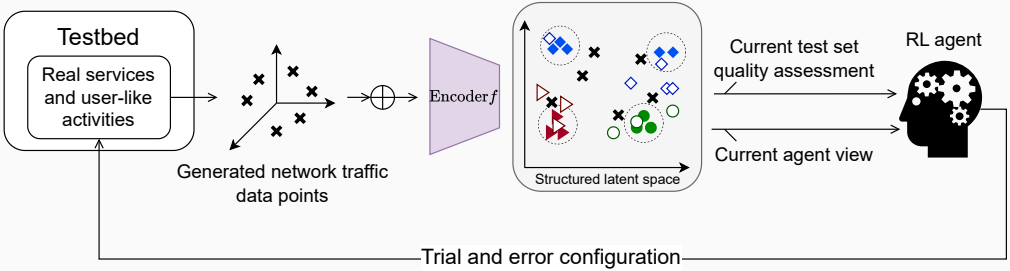
1. ~~GANs / diffusion models.~~
   - Synthetic generation may break protocol semantics $\Rightarrow$ unrealistic flows.
2. ~~Network simulators (e.g., NS-3, OMNeT++)~~
   - Models protocol interactions, offering more realism.
   - Remaining realism gap vs. real hardware and user behavior.
3. Using a **configurable live testbed**.
   - Best preserves realism of generated traffic.
   - Emits legitimate traffic across multiple protocols (HTTP/HTTPS, SSH, SMTP, DNS, ...) and can generate attack traffic under controlled conditions.

Which approaches can augment the test set?

1. ~~GANs / diffusion models.~~
   - Synthetic generation may break protocol semantics $\Rightarrow$ unrealistic flows.
2. ~~Network simulators (e.g., NS-3, OMNeT++)~~
   - Models protocol interactions, offering more realism.
   - Remaining realism gap vs. real hardware and user behavior.
3. Using a **configurable live testbed**.
   - Best preserves realism of generated traffic.
   - Emits legitimate traffic across multiple protocols (HTTP/HTTPS, SSH, SMTP, DNS, ...) and can generate attack traffic under controlled conditions.

Generate targeted configs so the testbed produces realistic flows targeting under-covered $\mathcal{Z}$ regions (low D, P, S).

Phase 2: Targeted Augmentation

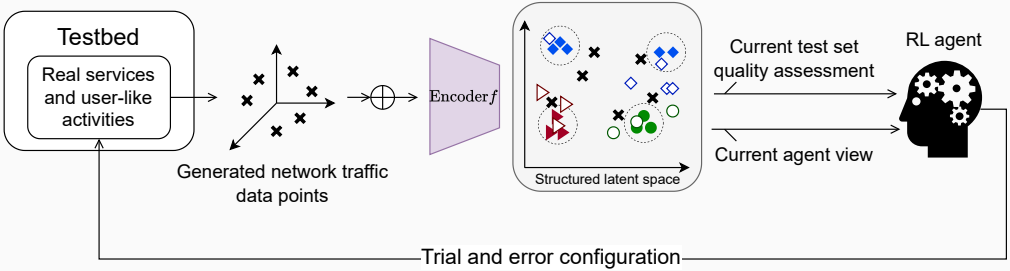


- Start from Preliminary & Phase 1 outputs
- **Training workflow**:

$$\boxed{\text{observe latent-space}} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowright \Rightarrow \pi^*$$

- Apply π^* to augment any $\mathcal{D}_{\text{test}}$

Phase 2: Targeted Augmentation

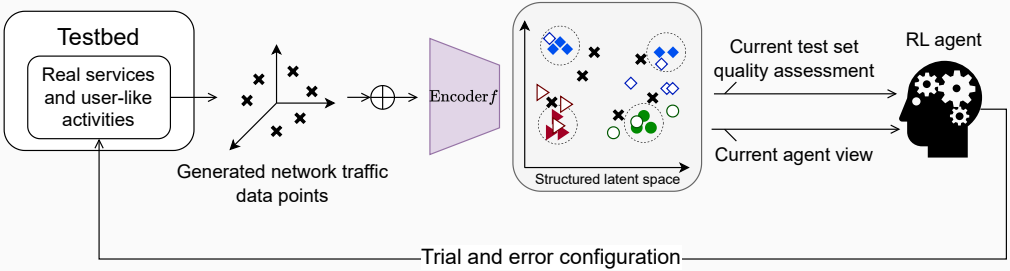


- Start from Preliminary & Phase 1 outputs
- **Training workflow**:

$$\text{observe latent-space} \rightarrow \boxed{\text{select a configuration}} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

- Apply π^* to augment any $\mathcal{D}_{\text{test}}$

Phase 2: Targeted Augmentation

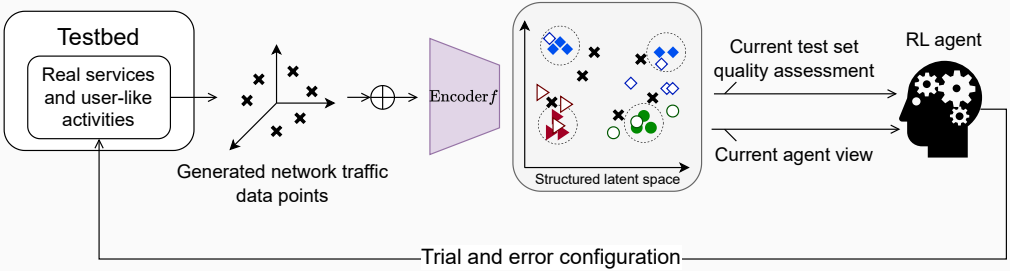


- Start from Preliminary & Phase 1 outputs
- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \boxed{\text{traffic generation}} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

- Apply π^* to augment any $\mathcal{D}_{\text{test}}$

Phase 2: Targeted Augmentation



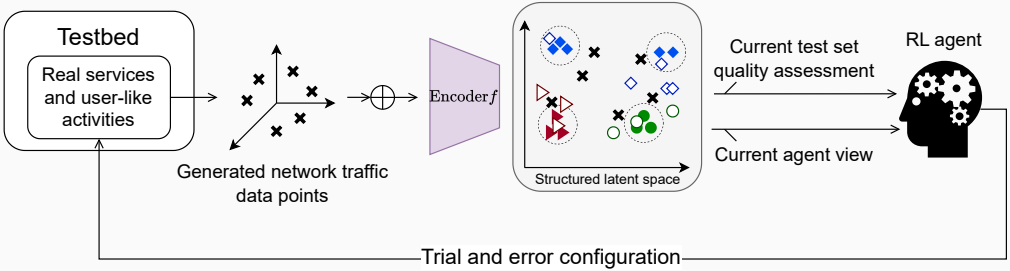
- Start from Preliminary & Phase 1 outputs
- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \boxed{\text{update } \mathcal{Z}_{\text{test}}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

- Apply π^* to augment any $\mathcal{D}_{\text{test}}$

Phase 2: Targeted Augmentation

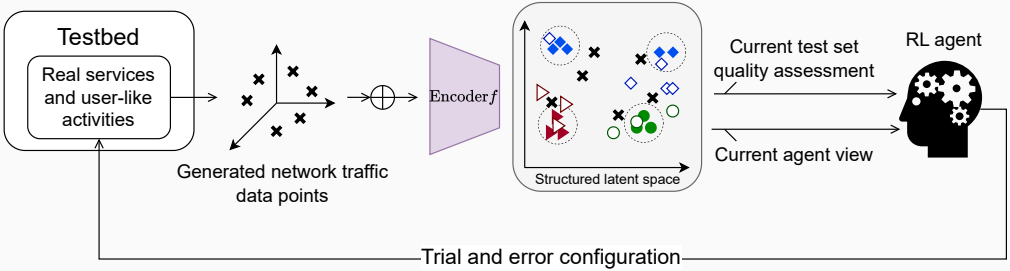


- Start from Preliminary & Phase 1 outputs
- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \boxed{\text{compute proximity, diversity, and scarcity}} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

- Apply π^* to augment any $\mathcal{D}_{\text{test}}$

Phase 2: Targeted Augmentation

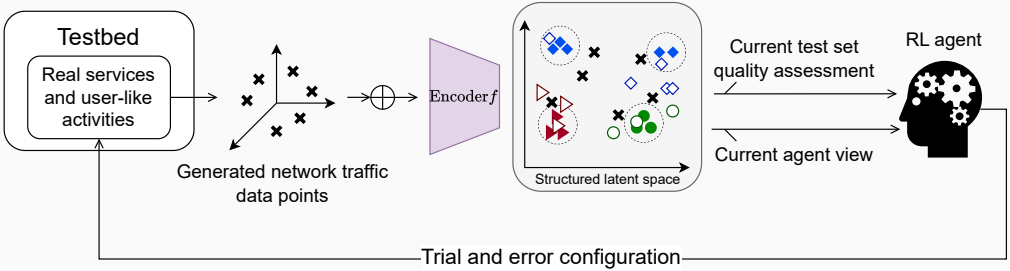


- Start from Preliminary & Phase 1 outputs
- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \boxed{\text{update policy } \pi} \circlearrowleft \Rightarrow \pi^*$$

- Apply π^* to augment any $\mathcal{D}_{\text{test}}$

Phase 2: Targeted Augmentation

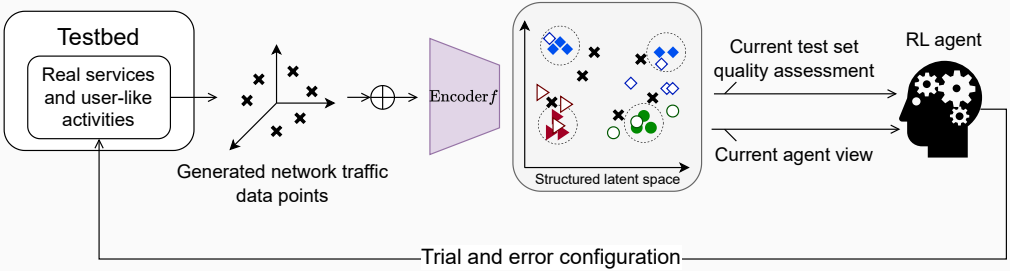


- Start from Preliminary & Phase 1 outputs
- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \; \boxed{\circlearrowright} \Rightarrow \pi^*$$

- Apply π^* to augment any $\mathcal{D}_{\text{test}}$

Phase 2: Targeted Augmentation



- Start from Preliminary & Phase 1 outputs
- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \boxed{\pi^*}$$

- Apply π^* to augment any $\mathcal{D}_{\text{test}}$

Experimental Setup

Datasets (60 / 20 / 20 split)

(Using the revised and corrected versions)

- **CIC-IDS 2017**: 0.93 M benign / 0.25 M attack flows
- **CSE-CIC-IDS 2018**: 59.8 M benign / 4.1 M attack flows

NIDS Models

- RF, SVM, 3-layer DNN (128-128-128 ReLU + Softmax)

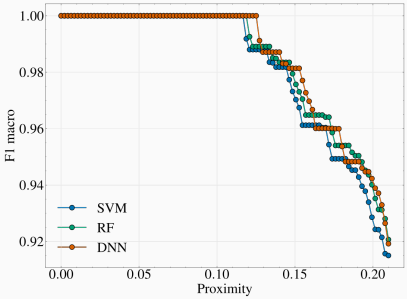
Agent Training

- Augmentation targets Benign SSH flows only

- Action space (client-side via `tc`): **Loss** $\in [5, 10]\%$ | **Jitter** $\in [4, 10]\,\mathrm{ms}$ | **Delay** $\in [10, 40]\,\mathrm{ms}$ | **Duplication** $\in [0.1, 5]\%$ | **Corruption** $\in [0.1, 10]\%$ | **Reordering** $\in [0.1, 50]\%$ | **Correlation** $\in [50, 100]\%$.

Proximity in Practice (IDS2017 only)

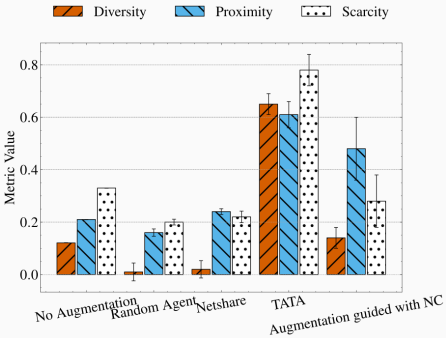
- Build 100 cumulative sub-tests by sorting z by distance to $N(z)$.
- Beyond a proximity threshold, macro-F1 decreases as points close to decision boundaries accumulate — **strong negative correlation**.
- Trends align across RF/DNN/SVM $\Rightarrow$ proximity is **model-agnostic**.



Macro-F1 vs. proximity across cumulative sub-tests (IDS2017).

Test-set Augmentation — Metrics

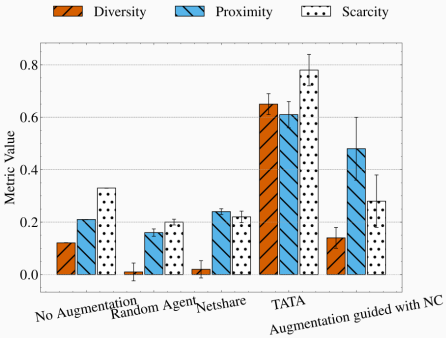
- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).



Test-set quality metrics (higher is more challenging/covered).

Test-set Augmentation — Metrics

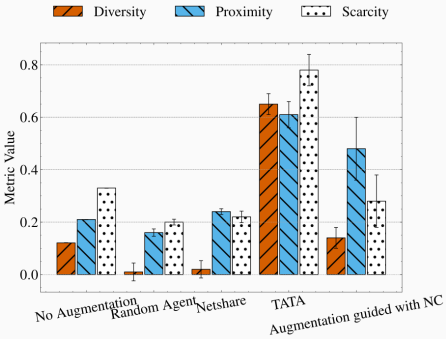
- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).
- **Quality metrics:**
  - **TATA:** $D + \sim 437\%$; $P + \sim 190\%$; $S + \sim 136\%$
  - **NC:** $D \approx 0$; $P + \sim 129\%$; $S \approx 0$
  - **Random/NetShare:** $D \downarrow$ to ≈ 0; $P \approx 0$; $S \downarrow$



Test-set quality metrics (higher is more challenging/covered).

Test-set Augmentation — Metrics

- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).
- **Quality metrics:**
  - **TATA:** $D + \sim 437\%$; $P + \sim 190\%$; $S + \sim 136\%$
  - **NC:** $D \approx 0$; $P + \sim 129\%$; $S \approx 0$
  - **Random/NetShare:** $D \downarrow$ to ≈ 0; $P \approx 0$; $S \downarrow$

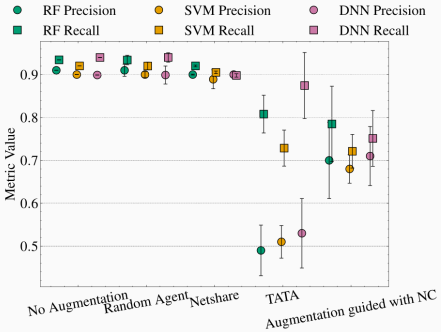


Test-set quality metrics (higher is more challenging/covered).

What is the impact on NIDS performance?

Test-set Augmentation — Impact on NIDS

- **NIDS impact (RF example):**
  - **TATA:** Precision $-\sim 46\%$; Recall $-\sim 13\%$
  - **NC:** Precision $-\sim 23\%$; Recall $-\sim 16\%$
- **Note:** higher scarcity spreads errors across boundaries $\Rightarrow$ FP $\uparrow \Rightarrow$ precision $\downarrow$.
- $\Rightarrow$ **proximity is the key driver;** scarcity and diversity are complementary (spread errors, avoid redundancy)



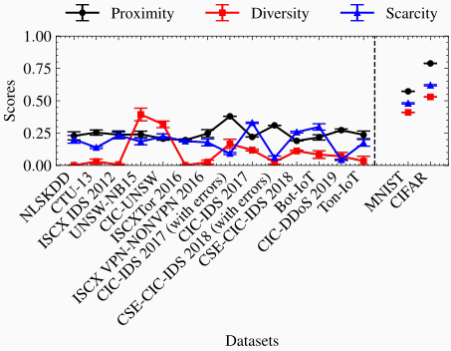
Impact on detection metrics (example: RF).

Datasets Analysis

- Apply D, P, S across many NIDS datasets (2009–2020), sorted by release year; averages over ten 60/20 splits.

- No increasing pattern; values mostly ≤ 0.5 with occasional peaks (e.g., early IDS2017/2018); low within-dataset variation.

- $\Rightarrow$ Despite richer, newer testbeds, NIDS test-set difficulty hasn't substantially increased. (CV refs like MNIST/CIFAR-10 score higher on D, P, S.)



Test-set assessment of NIDS datasets.

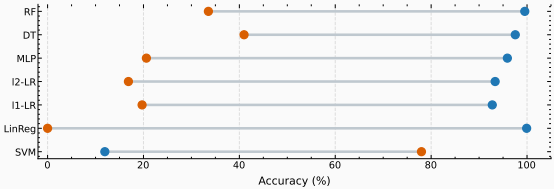
Part I conclusion

-
-
-

Part II: Auto-configuration

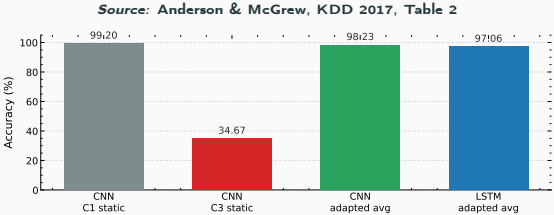
Why ML-based NIDS still need explicit adaptation

- ML-based NIDS are trained on historical traffic and deployed as static detectors.
- This assumes training and deployment traffic remain statistically close.
- In practice, benign traffic evolves, and attackers adapt.



Blue: Shifted benign traffic **Orange: Shifted malicious traffic**

Source: **Anderson & McGrew, KDD 2017, Table 2**



Source: **Qiao et al., IEEE TII 2022, Table V/VI and Section IV-C**

⇒ **Detection performance can degrade after deployment** ⇒ **explicit adaptation mechanisms are needed**

State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget

State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget
Drift detection	CADE, OWAD, TRANSCEND	Early warning of drift	Shift monitoring in data or outputs	None

State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget
Drift detection	CADE, OWAD, TRANSCEND	Early warning of drift	Shift monitoring in data or outputs	None
Online / continual learning	AOC-IDS, ENIDrift, ReCDA	Long-term tracking of evolving traffic	Incremental weight updates from traffic streams	Zero to low

State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget
Drift detection	CADE, OWAD, TRANSCEND	Early warning of drift	Shift monitoring in data or outputs	None
Online / continual learning	AOC-IDS, ENIDrift, ReCDA	Long-term tracking of evolving traffic	Incremental weight updates from traffic streams	Zero to low
Active / semi-supervised	Active-SVDD, HITL-IDS, METALS	Adaptation under scarce labels	Label queries and pseudo-label exploitation	Minimal

State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget
Drift detection	CADE, OWAD, TRANSCEND	Early warning of drift	Shift monitoring in data or outputs	None
Online / continual learning	AOC-IDS, ENIDrift, ReCDA	Long-term tracking of evolving traffic	Incremental weight updates from traffic streams	Zero to low
Active / semi-supervised	Active-SVDD, HITL-IDS, METALS	Adaptation under scarce labels	Label queries and pseudo-label exploitation	Minimal
Transfer / domain adaptation	ADA, NAEF, DI-NIDS	Deployment to a new network context	Knowledge transfer across domains	Zero to few

State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget
Drift detection	CADE, OWAD, TRANSCEND	Early warning of drift	Shift monitoring in data or outputs	None
Online / continual learning	AOC-IDS, ENIDrift, ReCDA	Long-term tracking of evolving traffic	Incremental weight updates from traffic streams	Zero to low
Active / semi-supervised	Active-SVDD, HITL-IDS, METALS	Adaptation under scarce labels	Label queries and pseudo-label exploitation	Minimal
Transfer / domain adaptation	ADA, NAEF, DI-NIDS	Deployment to a new network context	Knowledge transfer across domains	Zero to few
Meta-learning	FC-Net, PTN-IDS, RETSINA	Fast adaptation with limited data	Learned initialization or metric	Few to zero

State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget
Drift detection	CADE, OWAD, TRANSCEND	Early warning of drift	Shift monitoring in data or outputs	None
Online / continual learning	AOC-IDS, ENIDrift, ReCDA	Long-term tracking of evolving traffic	Incremental weight updates from traffic streams	Zero to low
Active / semi-supervised	Active-SVDD, HITL-IDS, METALS	Adaptation under scarce labels	Label queries and pseudo-label exploitation	Minimal
Transfer / domain adaptation	ADA, NAEF, DI-NIDS	Deployment to a new network context	Knowledge transfer across domains	Zero to few
Meta-learning	FC-Net, PTN-IDS, RETSINA	Fast adaptation with limited data	Learned initialization or metric	Few to zero
Threshold / score calibration	Bridges, Hundman, MAGPIE	Alert control without retraining	Threshold or score adjustment	None to very low

State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget
Drift detection	CADE, OWAD, TRANSCEND	Early warning of drift	Shift monitoring in data or outputs	None
Online / continual learning	AOC-IDS, ENIDrift, ReCDA	Long-term tracking of evolving traffic	Incremental weight updates from traffic streams	Zero to low
Active / semi-supervised	Active-SVDD, HITL-IDS, METALS	Adaptation under scarce labels	Label queries and pseudo-label exploitation	Minimal
Transfer / domain adaptation	ADA, NAEF, DI-NIDS	Deployment to a new network context	Knowledge transfer across domains	Zero to few
Meta-learning	FC-Net, PTN-IDS, RETSINA	Fast adaptation with limited data	Learned initialization or metric	Few to zero
Threshold / score calibration	Bridges, Hundman, MAGPIE	Alert control without retraining	Threshold or score adjustment	None to very low

Change monitoring only
No direct detector adaptation.

Model-targeted adaptation
Weights, transferred repr., learned init.

Threshold / HP adaptation
Post-training HP only; not model adaptation.

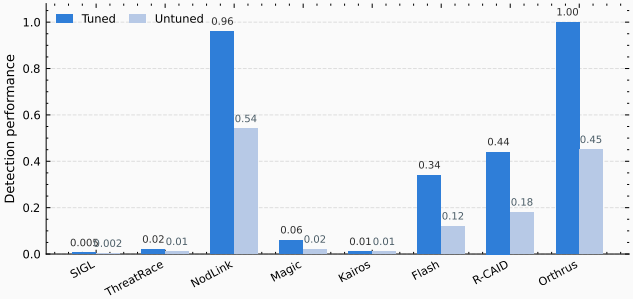
State of the art: main adaptation families

Family	Representative methods	Operational need	Adaptation mechanism	Label budget
Drift detection	CADE, OWAD, TRANSCEND	Early warning of drift	Shift monitoring in data or outputs	None
Online / continual learning	AOC-IDS, ENIDrift, ReCDA	Long-term tracking of evolving traffic	Incremental weight updates from traffic streams	Zero to low
Active / semi-supervised	Active-SVDD, HITL-IDS, METALS	Adaptation under scarce labels	Label queries and pseudo-label exploitation	Minimal
Transfer / domain adaptation	ADA, NAEF, DI-NIDS	Deployment to a new network context	Knowledge transfer across domains	Zero to few
Meta-learning	FC-Net, PTN-IDS, RETSINA	Fast adaptation with limited data	Learned initialization or metric	Few to zero
Threshold / score calibration	Bridges, Hundman, MAGPIE	Alert control without retraining	Threshold or score adjustment	None to very low

$\Rightarrow$ **Existing methods mainly adapt the model or its outputs, while train-time hyperparameters remain largely fixed.**

Is hyperparameter configuration important?

- Bilot et al. re-evaluate eight state-of-the-art PIDSs in a unified framework.
- Tuned and untuned variants are compared under the same task.
- Large performance gaps reveal strong hyperparameter sensitivity.
- Tuning therefore affects both detection performance and the fairness of empirical comparisons.



Source: **Bilot et al., USENIX Security 2025, Figure 5**

⇒ A deeper analysis of hyperparameter impact is provided in our paper *An Empirical Study on Configuration Robustness of Unsupervised IDS*, accepted to the IEEE/IFIP NOMS 2026 main track.

How is configuration handled today?

Conventional offline HPO

- Grid search, random search, Bayesian optimization, and evolutionary search.
- Repeated train-and-validate evaluations over candidate configurations.

Why not for ML-based NIDS adaptation?

These methods are static, offline, computationally expensive, and time-consuming procedures that must be re-run from scratch when the network context changes.

A NIDS-specific example

In Anser et al. (NOMS 2023), Bayesian optimization of a binary Random Forest NIDS used **300 iterations** for a single daily network context, which corresponds to about **50 minutes** when one HPO iteration takes up to **10 seconds**.

Refs: Bergstra & Bengio, JMLR 2012; Snoek et al., NeurIPS 2012; Anser et al., NOMS 2023.

Research question (RQ1)

Research question

How can we find a configuration of a ML-based NIDS in a timely manner while maintaining an acceptable level of detection performance, considering a heterogeneous network context?

From our analysis of hyperparameter impact

- A NIDS configuration method has to be applied at each network context change.
- A new network context is defined by a new attack in the network or by a topology change.

$\Rightarrow$ **Auto-configuration via meta-learning**

Auto-configuration via Meta-learning

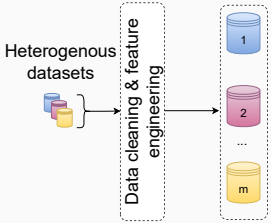
A. Meta-modeling

B. On the fly NIDS configuration

Trained NIDS

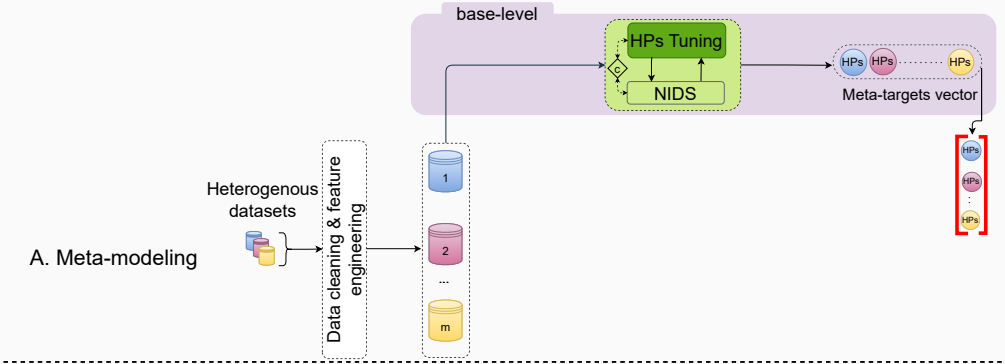
Auto-configuration via Meta-learning

A. Meta-modeling

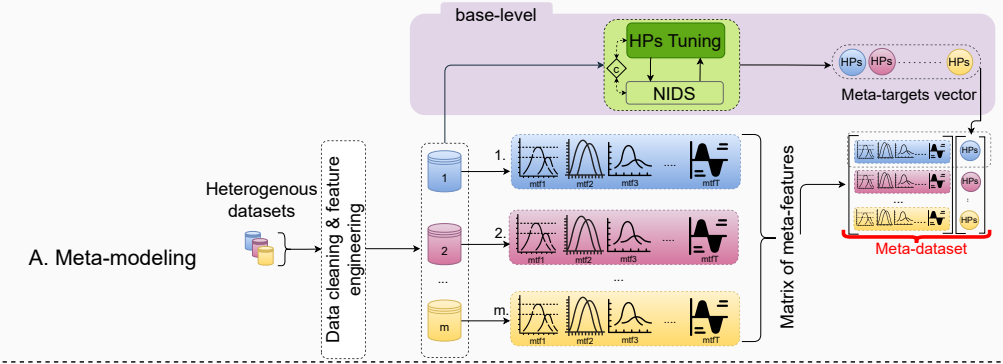


B. On the fly NIDS configuration

Auto-configuration via Meta-learning



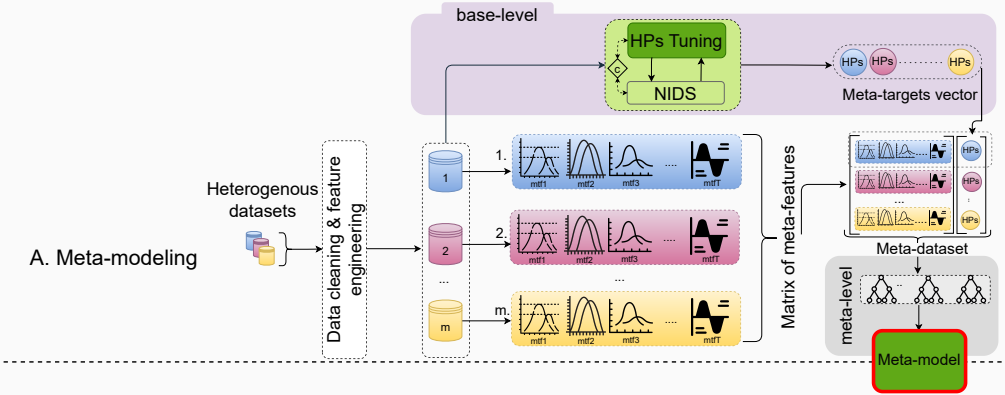
Auto-configuration via Meta-learning



B. On the fly NIDS configuration

Trained NIDS

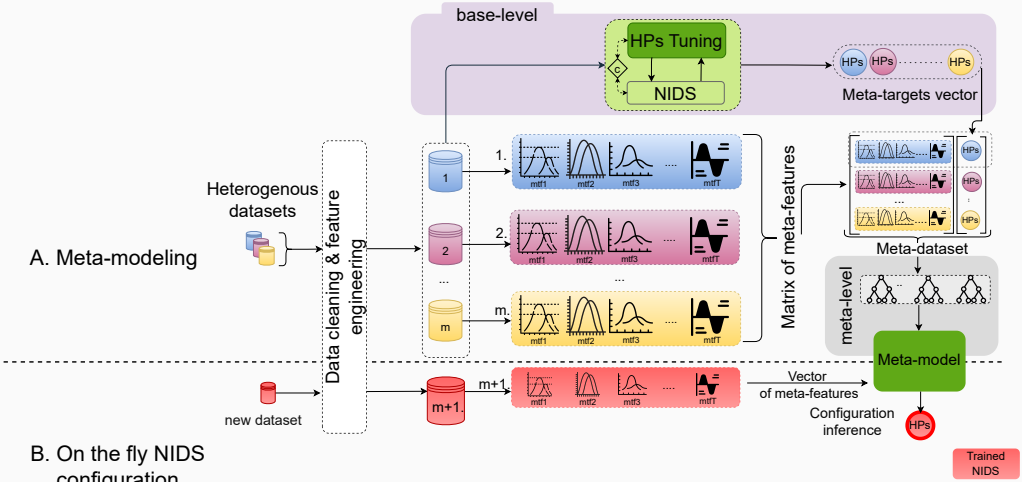
Auto-configuration via Meta-learning



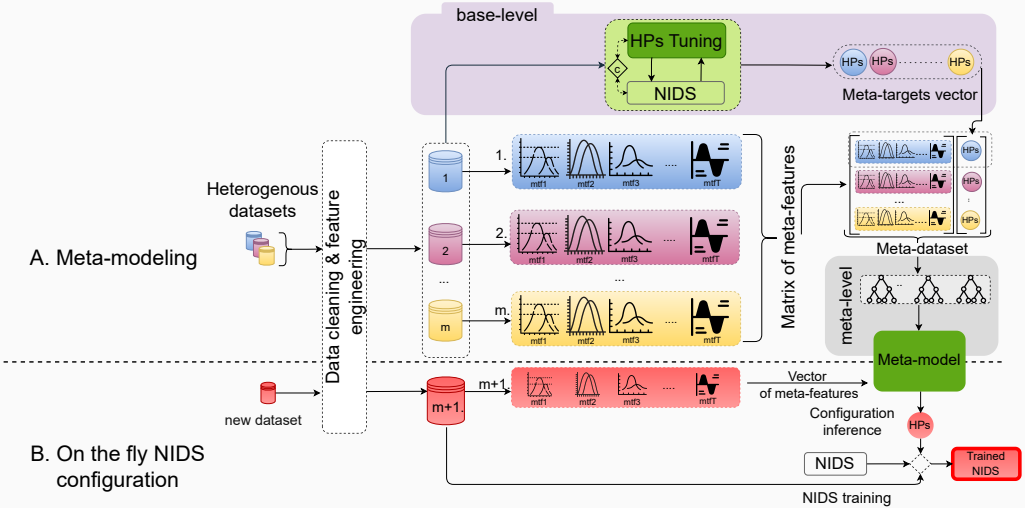
B. On the fly NIDS configuration

Trained NIDS

Auto-configuration via Meta-learning



Auto-configuration via Meta-learning



Questions?